

EVOLUTIONARY SCHEDULING + MULTI TASK LEARNING AS A BARGAINING GAME

Yash Patel, Archisa Bhattacharya



TABLE OF CONTENTS

01 WHAT IS MULTI-TASK LEARNING?

- a. Original problem of the paper
- b. Game Model

02 WHAT IS NASH BARGAINING?

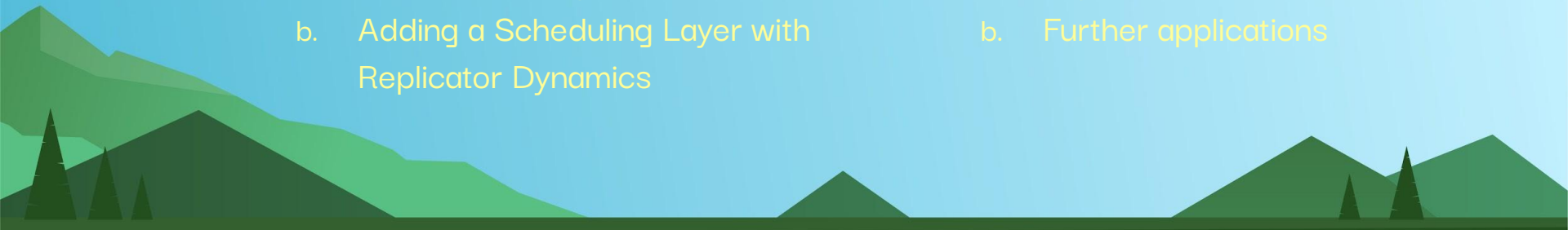
- a. Short Explanation of Nash Bargaining

03 WHAT WILL WE DO?

- a. The Need for Speed
- b. Adding a Scheduling Layer with Replicator Dynamics

04 WHO CARES ANYWAY?

- a. Expected outcomes
- b. Further applications



01

WHAT IS NASH BARGAINING?



CONCEPTUAL OVERVIEW: WHAT IS NASH BARGAINING?

Core Idea:

Nash Bargaining models how **two or more players negotiate over a shared decision** when cooperation benefits everyone, but each player also wants the best possible outcome for themselves.

Key Components:

- **Players:** Multiple tasks are negotiating with each other
- **Feasible Set (U):** All possible agreements
- **Disagreement Point (d):** What happens if no agreement is reached
- **Goal:** Find a fair, efficient solution that improves outcomes for all players beyond disagreement



Intuition:

Each player asks: “How much better off am I compared to if negotiations fail?”

The bargaining solution maximizes mutual benefit while maintaining fairness.

Source: [6.254 : Game Theory with Engineering Applications Lecture 14: Nash Bargaining Solution](#)

“One states as axioms several properties that would seem natural for the solution to have and then one discovers that axioms actually determine the solution uniquely.”

—JOHN NASH

- A **bargaining problem** is a pair (U, d) where $U \subset \mathbb{R}^2$ and $d \in U$. We assume that
 - U is a convex and compact set.
 - There exists some $v \in U$ such that $v > d$ (i.e., $v_i > d_i$ for all i).
- We denote the set of all possible bargaining problems by $\mathcal{B}$.
- A **bargaining solution** is a function $f : \mathcal{B} \rightarrow U$.
- We will study bargaining solutions $f(\cdot)$ that satisfy a list of reasonable axioms.

NASH'S FOUR CORE AXIOMS

Nash proved that a fair bargaining solution should satisfy:

1. **Pareto Efficiency** No possible agreement should leave resources unused.

$$f_1(U,d) = f_2(U,d)$$

Meaning: No player can improve without reducing another player's payoff.

2. **Symmetry** If players are identical, outcomes should be equal.

$$f_1(U,d) = f_2(U,d)$$

Meaning: Equal players $\rightarrow$ equal outcomes.

3. **Invariance to Utility Scaling** Changing utility measurement units should not affect the solution

$$f_i(U',d') = \alpha_i f_i(U,d) + \beta_i$$

Meaning: Fairness remains unchanged under rescaling.

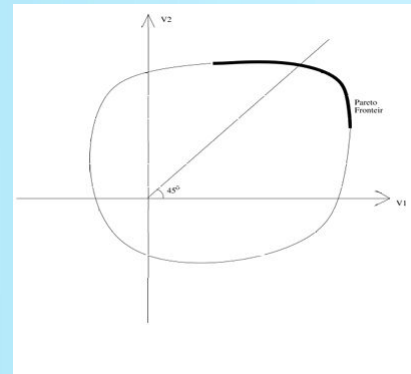
4. **Independence of Irrelevant Alternatives** Removing unused options shouldn't change chosen solution.

If $U' \subseteq U$ and $f(U,d) \in U'$, then:

$$f(U',d) = f(U,d)$$

Meaning:

Irrelevant alternatives do not matter.



Proposition

Nash bargaining solution $f^N(U,d)$ is the unique bargaining solution that satisfies the 4 axioms.

NASH BARGAINING SOLUTION - MATHEMATICAL FORMULATION

Utility vector:

$$v = (v_1, v_2)$$

Disagreement point:

$$d = (d_1, d_2)$$

Purpose: Maximizes mutual gain over disagreement, in order to promote fairness, cooperation, and balanced optimization

Nash Bargaining Solution

Definition

We say that a pair of payoffs (v_1^*, v_2^*) is a **Nash bargaining solution** if it solves the following optimization problem:

$$\begin{aligned} \max_{v_1, v_2} \quad & (v_1 - d_1)(v_2 - d_2) \\ \text{subject to} \quad & (v_1, v_2) \in U \\ & (v_1, v_2) \geq (d_1, d_2) \end{aligned} \tag{1}$$

We use $f^N(U, d)$ to denote the Nash bargaining solution.

Remarks:

- *Existence of an optimal solution:* Since the set U is compact and the objective function of problem (1) is continuous, there exists an optimal solution for problem (1).
- *Uniqueness of the optimal solution:* The objective function of problem (1) is strictly quasi-concave. Therefore, problem (1) has a unique optimal solution.

- It can be seen that this game has a unique SPE in which, Player 1 proposes $\hat{x}$ and accepts an offer y if and only if $y_1 \geq \hat{y}_1$. Player 2 proposes $\hat{y}$ and accepts an offer x if and only if $x_1 \geq \hat{x}_1$, where

$$\hat{x}_1 = \frac{1 - d_2 + (1 - \alpha)d_1}{2 - \alpha}, \quad \hat{y}_1 = \frac{(1 - \alpha)(1 - d_2) + d_1}{2 - \alpha}$$

- Letting $\alpha \rightarrow 0$, we have $\hat{x}_1 \rightarrow 1/2 + 1/2(d_1 - d_2)$, which coincides with the Nash bargaining solution.



02

WHAT IS MULTI-TASK LEARNING?

AN ANALOGY: HIKERS ON A MOUNTAIN

Imagine a group of hikers tied together, trying to climb the same mountain.

Each hiker represents a task. Each hiker wants to move in the direction that helps *their own climb* the most.

But the rope represents the shared model parameters. So the group cannot move in every direction at once.

If one hiker pulls too hard, the whole group may move in a direction that helps that hiker but hurts the others.

That is the multi-task gradient conflict problem.

Where Nash Bargaining Comes In

Nash Bargaining acts like a fair negotiation before the group moves. Instead of letting the strongest hiker dominate, the hikers agree on a direction that gives everyone some improvement.

So the final update is not:

“Which task pulls hardest?”

It becomes:

“Which direction gives the fairest shared progress?”



MULTI-TASK LEARNING AS A BARGAINING GAME

Abstract

In Multi-task learning (MTL), a joint model is trained to simultaneously make predictions for several tasks. Joint training reduces computation costs and improves data efficiency; however, since the gradients of these different tasks may conflict, training a joint model for MTL often yields lower performance than its corresponding single-task counterparts. A common method for alleviating this issue is to combine per-task gradients into a joint update direction using a particular heuristic. In this paper, we propose viewing the gradients combination step as a bargaining game, where tasks negotiate to reach an agreement on a joint direction of parameter update. Under certain assumptions, the bargaining problem has a unique solution, known as the *Nash Bargaining Solution*, which we propose to use as a principled approach to multi-task learning. We describe a new MTL optimization procedure, Nash-MTL, and derive theoretical guarantees for its convergence. Empirically, we show that Nash-MTL achieves state-of-the-art results on multiple MTL benchmarks in various domains.

Multi-Task Learning as a Bargaining Game

Aviv Navon^{*1} Aviv Shamsian^{*1} Idan Achituve¹ Haggai Maron² Kenji Kawaguchi³
Gal Chechik^{1,2} Ethan Fetaya¹

Abstract

In Multi-task learning (MTL), a joint model is trained to simultaneously make predictions for several tasks. Joint training reduces computation costs and improves data efficiency; however, since the gradients of these different tasks may conflict, training a joint model for MTL often yields lower

2000) and in practice (e.g., auxiliary learning, Liu et al., 2019a; Achituve et al., 2021; Navon et al., 2021a).

Unfortunately, MTL often causes performance degradation compared to single-task models (Standley et al., 2020). A main reason for such degradation is gradients conflict (Yu et al., 2020a; Wang et al., 2020; Liu et al., 2021a). These

<https://arxiv.org/pdf/2202.01017>

Link to paper

<https://github.com/AvivNavon/nash-mtl>.

Link to Git

GAME MODEL

Type of Game: Cooperative bargaining game

Players:

- The tasks in the multi-task learning system
- Each task acts like a player because it has its own goal and loss function
- The players are not separate models. The players are the tasks

Strategy space: Tasks bargain over the shared update direction

Utility: Each task's utility measures how much it benefits from the shared update

- utility = how aligned the update is with that task's gradient



WHAT IS MULTI-TASK LEARNING, ANYWAY?

- **Multi-task learning (MTL):** ML practice where multiple learning tasks are solved at the same time
- The base setup is a shared neural network, where each task is a different prediction problem using the same input and shared model
- Each task produces a gradient showing how it wants the model to update, but these gradients can conflict, meaning an update that helps one task may hurt another
- The issue is that each task wants the model to update in a way that improves its own performance
- **Main idea:** tasks “negotiate” over the shared update so the final direction is fair and useful for all tasks
- **What was done before?** Why is Nash Bargaining Different? Most MTL optimization algorithms compute gradients for all tasks, combine them into a joint direction using an aggregation function, and update with a single-task optimization algorithm. Nash Bargaining allows the algorithm to find a pareto optimal direction first.

The background is a stylized landscape. On the left, there are several green mountains of varying heights and shades of green. In the foreground, there are dark green silhouettes of coniferous trees. On the right, a large yellow sun is partially visible in the top right corner. The sky is a light blue gradient.

03

**OUR
IMPLEMENTATION**

EVOLUTIONARY SCHEDULING FOR NASH BARGAINING-BASED MULTI-TASK LEARNING

Problem: Traditional multi-task learning struggles because multiple tasks share one model while producing conflicting gradients.

Common issues:

- **Task domination:** stronger tasks can overpower weaker ones, causing imbalance in learning
- **Gradient conflict:** task objectives may push shared parameters in opposing directions
- **Poor fairness:** some tasks consistently benefit more than others
- **Weak adaptability:** static weighting struggles as task difficulty changes over time
- **Static prioritization:** task importance remains relatively fixed instead of evolving strategically

Existing approach: Nash-MTL - Models tasks as bargaining players - Produces fair gradient updates

Key limitation: - Fairness is optimized step-by-step - Task priority does not adapt strategically over time

Proposed contribution: Introduce a replicator dynamics-based evolutionary scheduler before Nash bargaining

EVOLUTIONARY SCHEDULER PIPELINE

Dynamic Payoff Matrix Formulation

Dynamic payoff matrix:

$$A_{ij}(t) = \alpha_1 u_i(t) + \alpha_2 c_i(t) + \alpha_3 s_{ij}(t)$$

Task urgency:

$$u_i(t) = \frac{1}{\epsilon + \max(p_i(t), 0)}$$

Task improvement:

$$p_i(t) = L_i(t - 1) - L_i(t)$$

Gradient conflict:

$$c_i(t) = \frac{1}{K-1} \sum_{j \neq i} \max(0, -\cos(g_i, g_j))$$

Gradient similarity:

$$s_{ij}(t) = \max(0, \cos(g_i, g_j))$$

Interpretation

- $u_i(t)$: urgency of task i based on lack of improvement
- $c_i(t)$: degree of conflict between task i and other tasks
- $s_{ij}(t)$: cooperative compatibility between tasks i and j
- $\alpha_1, \alpha_2, \alpha_3$: tunable coefficients controlling contribution of urgency, conflict, and synergy

Task fitness:

$$f_i(t) = (A(t)w(t))_i$$

This means each task's evolutionary fitness depends on:

- Dynamic urgency
- Gradient conflict
- Cooperative gradient alignment
- Current task-weight distribution

Average population fitness:

$$\bar{f}(t) = w(t)^T A(t)w(t)$$

Replicator dynamics update:

$$w_i(t+1) = w_i(t) \frac{f_i(t)}{\bar{f}(t)}$$

Weighted Nash bargaining:

$$\tilde{g}_i = w_i(t)g_i$$

$$\Delta\theta = NashMTL(\tilde{g}_1, \dots, \tilde{g}_K)$$

Full Framework

Task Losses + Gradient Signals $\rightarrow A(t) \rightarrow f_i(t) \rightarrow w_i(t+1) \rightarrow \tilde{g}_i \rightarrow \text{Nash Bargaining} \rightarrow \Delta\theta$

Summary

This formulation transforms task prioritization into a dynamic evolutionary game where:

- Training signals act as external influences
- Payoffs evolve over time
- Replicator dynamics adjusts bargaining power
- Nash bargaining ensures fair final gradient aggregation

This makes the framework mathematically aligned with both:

- Dynamic Influence Replicator Evolution
- Nash-MTL

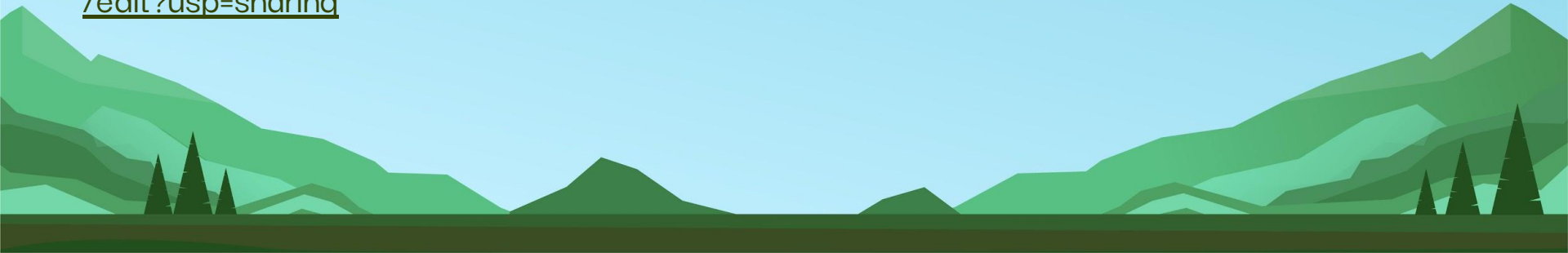
INITIAL IMPLEMENTATION IDEA & POTENTIAL APPLICATIONS

LINK TO GOOGLE DOC OF OUR INITIAL IMPLEMENTATION IDEA:

<https://docs.google.com/document/d/1x8VSO5P2btIKq-nFYY7XW3J1VDMDTB-HMk24cbmye4Q/edit?usp=sharing>

LINK TO GOOGLE DOC OF POTENTIAL APPLICATIONS OF OUR IMPLEMENTATION:

https://docs.google.com/document/d/1AesGNHWEh6Q3MklmrA1cv1v83JMT_di4ZUOORrjHqCA/edit?usp=sharing



THANKS



CREDITS: This presentation template was
created by [Slidesgo](#) including icons by [Flaticon](#),
infographics & images by [Freepik](#)

Please keep this slide for attribution.

